



PROJECT RAKSHAK

Adaptive Disaster Intelligence Platform

Optimising Emergency Response Efficiency

Arnav Chauhan

Mechanical Engineering, IIT Kharagpur
arnavch246@gmail.com

Shaunak Majumdar

Mechanical Engineering, IIT Kharagpur
onlyshaun17@gmail.com

Abstract

This report outlines the development of **Project Rakshak**, an adaptive multi-agent system to optimize emergency response efficiency. By leveraging multimodal retrieval and historical context, the platform grounds crisis decisions in evidence to maximize resource impact while ensuring explainable outcomes.

Contents

1	Description of the Problem Statement	2
2	The Proposed Solution	3
3	System Architecture	4
3.1	High-Level Architecture Overview	5
3.2	Agent Breakdown	5
4	Why Qdrant?	5
4.1	Native Multimodal Vector Support	5
4.2	Hybrid Search with Server-Side Fusion	5
4.3	Binary Quantization for Scale	5
4.4	Rich Payload Schema and Geospatial Indexing	6
4.5	Recommendation API	6
4.6	Long-Term Persistent Episodic Memory	6
5	Methodology	6
5.1	Multimodal Perception	6
5.2	Memory Architecture	6
5.3	Reasoning and Decision Making	7
6	Implementation Highlights	7
6.1	Performance and Qualitative Results	7
6.2	Qualitative Outputs	7
7	Ethics, Safety and Novelty	9
8	Conclusion	10
9	Future Scope	10
A	Agent Breakdown	11
B	Implementation & Reproducibility	12
B.1	Methodology Details: Datasets	12
B.2	Technology Stack	12
B.3	Execution Instructions	12
C	Qdrant Collections Structure	12
D	Research-Grade Technical Glossary	13
D.1	Embeddings & Representation Learning	13
D.2	Fusion Retrieval Algorithms	14
D.3	HNSW (Hierarchical Navigable Small World)	14

1 Description of the Problem Statement

Rapid, transparent, and trustworthy damage assessment remains a critical bottleneck in large-scale disaster response. Despite significant advances in sensing technologies and analytics, emergency response teams still rely heavily on manual GIS workflows, fragmented sensor feeds, and incomplete situation reports. These constraints lead to systemic inefficiencies during time-critical operations.

Key Challenges

- **Slow Triage and Decision Latency:** Translating raw satellite imagery and incoming citizen distress signals into actionable priorities often takes hours, delaying the deployment of scarce rescue and medical resources.
- **Underutilization of Historical Experience:** Large, curated archives of disaster imagery and annotations (e.g., the xBD dataset) exist, yet are rarely reused in operational settings. As a result, each new disaster is effectively treated as a first-time event, ignoring valuable historical precedent.
- **Unstructured Human Distress Signals:** Victim reports received via chat messages and voice calls are noisy, emotionally charged, and geographically dispersed, making it difficult to fuse them into a coherent, decision-ready operational picture.

Problem Objective

The objective of this work is to design and implement a *multi-agent disaster intelligence platform* that addresses these challenges by grounding real-time response decisions in historical memory and multimodal evidence.

Specifically, the proposed system is designed to:

- Leverage **historical imagery and annotations from over 19 major disasters** as a persistent, episodic memory.
- Accept **minimal input** for a new incident, consisting primarily of:
 - A single post-disaster satellite image,
 - Latitude and longitude coordinates,
 - Optional disaster-type metadata,
 - Optional victim distress signals (text or voice).
- Perform **hybrid multimodal retrieval** across visual, textual, geographic, and categorical dimensions to identify relevant historical precedents.
- Generate a **comprehensive and explainable damage assessment report**, including:
 - Severity classification and spatial damage distribution,
 - Cited references to similar historical incidents and learned patterns,
 - A triage recommendation with resource allocation and estimated response timelines,
 - A unified distress and urgency summary synthesized from live victim chat and voice signals.

Design Constraints

The system must function as a solution for *high societal impact*: architected as a set of cooperating agents, grounded in a real vector database (Qdrant, scalable via **Qdrant Cloud**), and capable of delivering rapid, life-saving intelligence in real-world disaster scenarios. At the same time, it must reflect state-of-the-art techniques in multimodal retrieval, memory-augmented reasoning, and explainable artificial intelligence.

2 The Proposed Solution

We propose **Project Rakshak**, a **15-agent, multi-layer disaster intelligence system along with a specialized distress call agent** that transforms historical disaster imagery, metadata, and live distress signals into a unified, explainable emergency response recommendation. The core idea is to treat past disasters as *episodic memories* stored in a *Qdrant vector database*, and to retrieve and reason over these memories using *multimodal hybrid search* efficiently using *multi-stage agent-based reasoning* as well as a specialised distress call/chat agent connected to the main pipeline.

Given a new post-disaster satellite image and its geographic coordinates, the system operates as follows:

1. **Multimodal Encoding and Storage:** The system extracts **DINOv2 visual embeddings** from satellite imagery datasets, **CLAP acoustic embeddings** from emergency audio logs, and **BM25-style sparse embeddings** from structured metadata. These representations are upserted into Qdrant as *named multimodal vectors* (image, audio, and text) with rich payload metadata.
2. **Hybrid Retrieval over Historical Memory:** A hybrid search pipeline leverages Qdrant's **prefetch mechanism**, **Reciprocal Rank Fusion (RRF)**, and binary quantization to retrieve similar historical incidents efficiently. Disaster-type-aware filters and cross-disaster transfer penalties ensure relevance and reduce domain noise.
3. **Evidence-Based Reasoning and Reporting:** A dedicated reasoning layer:
 - Synthesizes severity distributions and building-level damage statistics using retrieved evidence to help us understand the current situation from the image.
 - Generates a **three-stage LLM report** to combine the current situation with historical context:
 1. **Recommendation :** Identifies visually similar damage patterns using Qdrant's **Recommendation engine** and retrieves geographically proximal historical disasters via Qdrant's **Geo-Radius API**.
 2. **Content Summarizer:** Synthesizes the current incident data with retrieved historical precedents into a coherent narrative, grounding every claim in cited evidence.
 3. **Auditor/Evaluator:** Verifies factual consistency, removes hallucinations, and ensures all statements are traceable to retrieved records.
 - Calibrates confidence scores and produces visual explanations (Severity charts, Confidence gauges, Geo-similarity maps).
4. **Integration of Stress Signals:** A VictimChat pipeline processes live chat and voice inputs using audio embeddings of 911 datasets inside Qdrant's collections to detect panic levels.

The agent performs PII-safe interaction with victims and converts unstructured distress signals into structured triage indicators generating emergency reports.

5. **Unified Triage Recommendation:** Outputs from the imagery-driven *Rakshak Intel* pathway and the chat/voice-driven *Distress Mode* are fused into a single triage report recommending priority level, resource allocation, and estimated response time.

The system is implemented in Python using modular agents and layered orchestration, with Qdrant serving as the long-term memory substrate. Project Rakshak demonstrates how *multi-modal retrieval*, *structured memory*, and *multi-agent coordination* can deliver competition-level, explainable disaster intelligence.

3 System Architecture

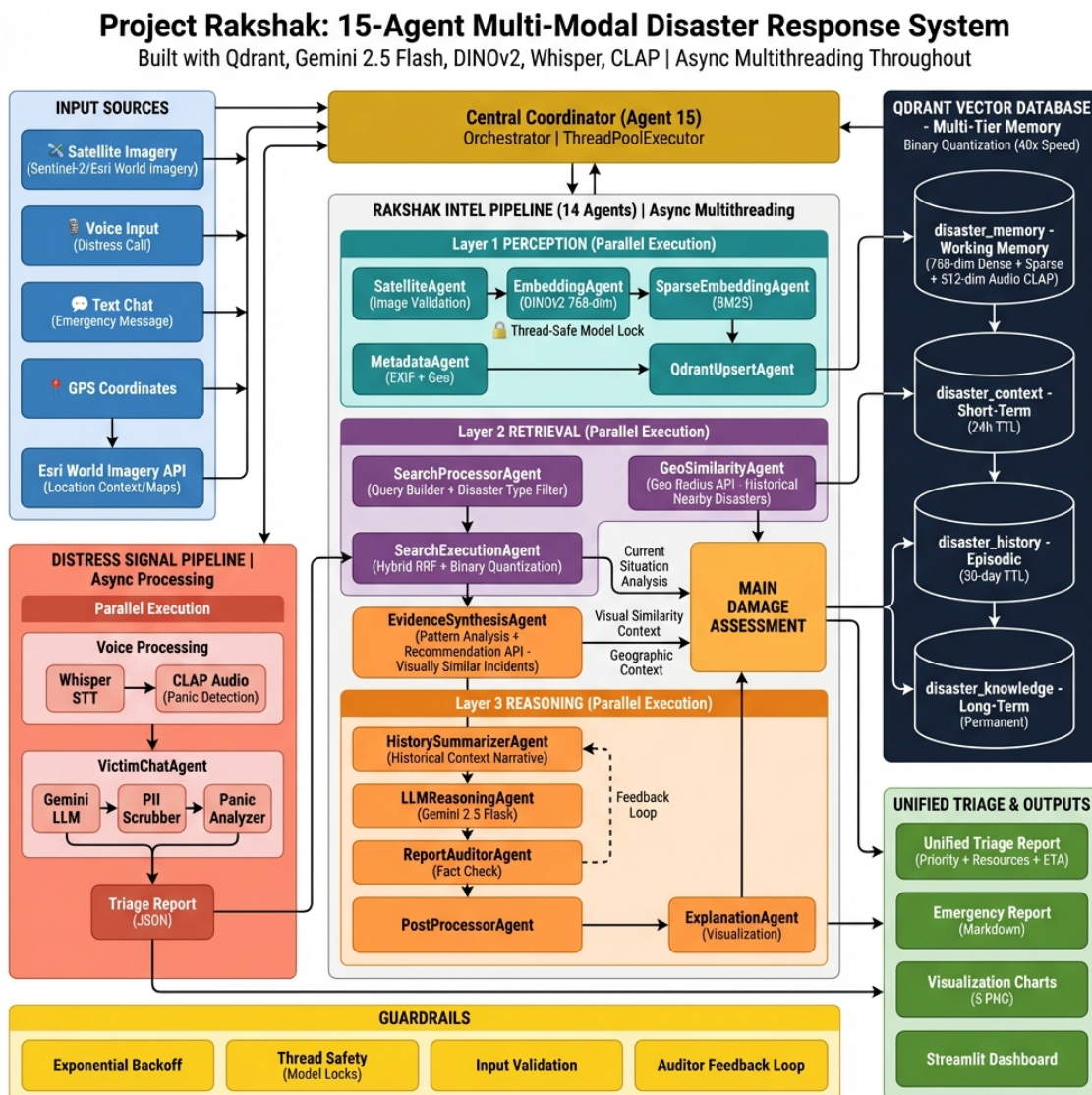


Figure 1: High-level architecture of Project Rakshak, showing the 3-layer agent design (Perception, Retrieval, Reasoning) and the parallel Distress Signal Subsystem.

The proposed platform is architected as a modular, multi-agent system intentionally decomposed into **three semantic layers** and **15 cooperating agents**. In addition, a parallel **Distress Signal Subsystem** operates alongside the main pipeline to incorporate real-time human inputs.

3.1 High-Level Architecture Overview

At a high level, the system follows a layered design that separates perception, retrieval, and reasoning concerns.

- **Layer 1 – Perception and Ingestion:** Ingests satellite imagery, computes dense/sparse embeddings, parses metadata, and upserts into Qdrant.
- **Layer 2 – Retrieval and Search:** Handles intelligent query planning, hybrid multimodal retrieval (dense, sparse, metadata), and result validation.
- **Layer 3 – Reasoning and Post-Processing:** Synthesizes evidence, performs multi-stage LLM reasoning, calibrates confidence, and produces triage recommendations.
- **Central Coordinator:** The DAG orchestrator Organising agent execution and data flow that utilizes **multithreading** to manage data flow and parallel execution of the 15-agent pipeline.

Distress Signal Subsystem. A dedicated subsystem processes live victim chat and voice inputs. This subsystem captures geolocation, textual, and acoustic signals; performs geo-aware fast-path risk checks; interacts with victims through a Gemini-based emergency persona; and extracts structured triage signals.

3.2 Agent Breakdown

A detailed breakdown of the 15 agents and their specific responsibilities is provided in **Appendix A**.

4 Why Qdrant?

4.1 Native Multimodal Vector Support

The primary collection, `disaster_memory`, leverages Qdrant's **Named Vectors** capability to store heterogeneous representations within a single point (Dense Image + Sparse Text + Audio). This enables tight fusion without external score normalization.

4.2 Hybrid Search with Server-Side Fusion

We use multiple **Prefetch** queries fused via **Reciprocal Rank Fusion (RRF)**, avoiding brittle application-layer normalization and yielding stable rankings.

4.3 Binary Quantization for Scale

We explicitly enable **Binary Quantization** on the image vector, offering $\sim 40\times$ faster search and $\sim 32\times$ reduction in RAM. By compressing high-dimensional floating-point vectors into compact binary strings, we achieve sub-millisecond retrieval speeds even effectively on CPU-bound edge devices. We use **Oversampling and Rescoring** to recover precision, ensuring that the speed advantage does not come at the cost of retrieval accuracy. This architectural efficiency

ensures that Project Rakshak can be seamlessly deployed on **Qdrant Cloud** for nation-scale disaster management.

4.4 Rich Payload Schema and Geospatial Indexing

Qdrant's payload system enables fast **geo-radius filtering** for regional priors and response safety zones. By indexing geospatial coordinates alongside vector embeddings, we can effectively filter for the *historical context* of disasters that have occurred in the nearby region. This allows the system to prioritize locally relevant data, such as past flood patterns or seismic activity history in the user's vicinity, significantly improving the relevance of the threat assessment.

4.5 Recommendation API

We invoke Qdrant's **Recommendation API** to perform "more-like-these" expansion over top-confidence incidents, reducing query drift. This is crucial for visual data; checking for *visually similar* disaster imagery efficiently allows the system to rapidly identify common structural damage patterns or environmental changes across different events without needing massive labeled datasets.

4.6 Long-Term Persistent Episodic Memory

Project Rakshak employs Qdrant as a **long-term persistent episodic memory** to store and manage historical disaster events in a durable and scalable manner. Unlike transient, in-memory indexing mechanisms, the system relies on **disk-backed persistence**—enabled through memory-mapped storage (`mmap`) and write-ahead logging (WAL)—to ensure the permanent retention of rich multimodal representations derived from over 19 major disaster scenarios.

This persistent memory layer enables the system to continuously ground real-time inference and decision-making in historical evidence. By leveraging previously observed catastrophe patterns, Project Rakshak enhances situational awareness and improves the effectiveness and reliability of future disaster response strategies.

5 Methodology

The proposed system follows a memory-augmented, multi-agent methodology.

5.1 Multimodal Perception

Visual Embeddings (Dense): `EmbeddingAgent` encodes tiles (224×224 px) using **DINOv2** into 768-dim vectors. **Text/Metadata (Sparse):** `SparseEmbeddingAgent` encodes metadata into BM25 sparse vectors. **Audio:** `AudioIngestionAgent` (Whisper + CLAP) transcribes audio and infers panic levels.

5.2 Memory Architecture

Qdrant serves as a **structured, multi-tier long-term memory**:

1. **Episodic Memory:** `disaster_memory` (19+ major events).
2. **Acoustic Memory:** `disaster_audio` (Distress signals).

5.3 Reasoning and Decision Making

The reasoning layer integrates retrieval, synthesis, explanation, and triage. Retrieval uses parallel strict/broad/geo searches fused via RRF. Synthesis computes distributions. Reasoning uses a three stage LLM pipeline (Communicator → Summarizer → Auditor).

6 Implementation Highlights

6.1 Performance and Qualitative Results

Pipeline Stage	Approx. Latency
Visual Embedding Extraction	50–100 ms per tile
Qdrant Hybrid Search (BQ)	20–80 ms
Evidence Refinement	< 10 ms
LLM Report Generation	1–3 seconds

Table 1: System performance metrics on standard hardware.

Retrieval Quality: Hybrid retrieval consistently retrieves incidents with similar visual patterns (e.g., specific flood types) and respects disaster-type filters.

Distress Integration: High panic signals from voice/chat successfully trigger triage overrides, escalating priority levels automatically.

6.2 Qualitative Outputs

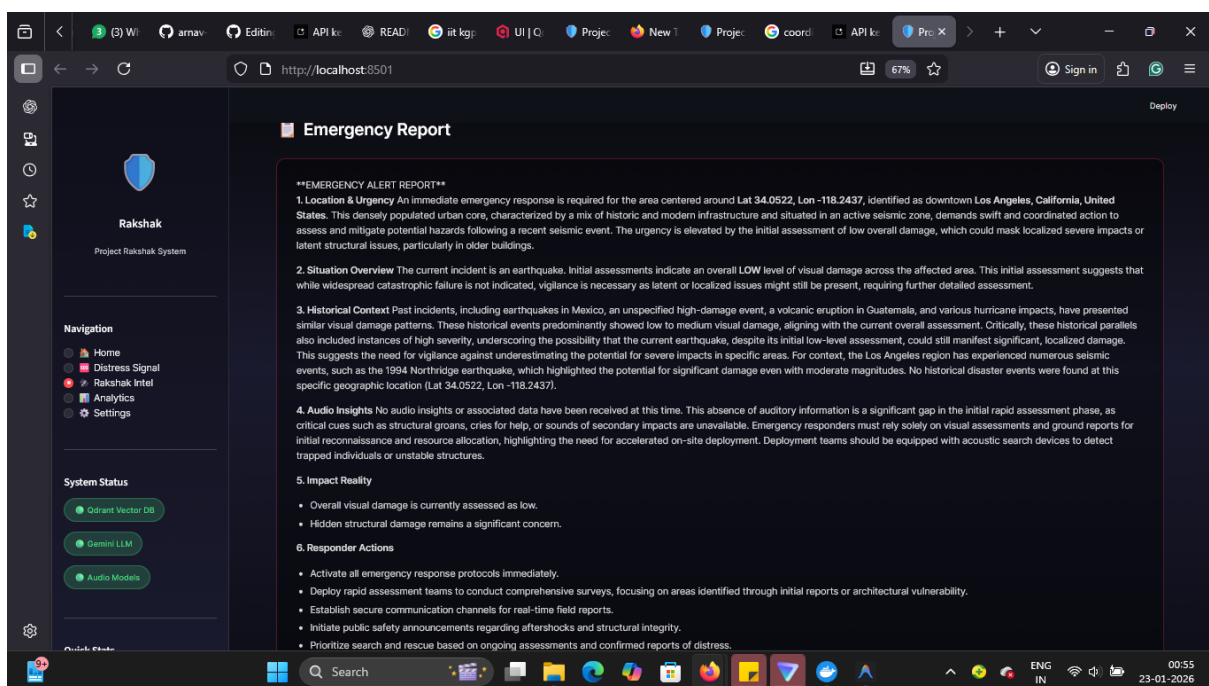


Figure 2: Damage Assessment Analysis: DINOv2-based severity mapping.

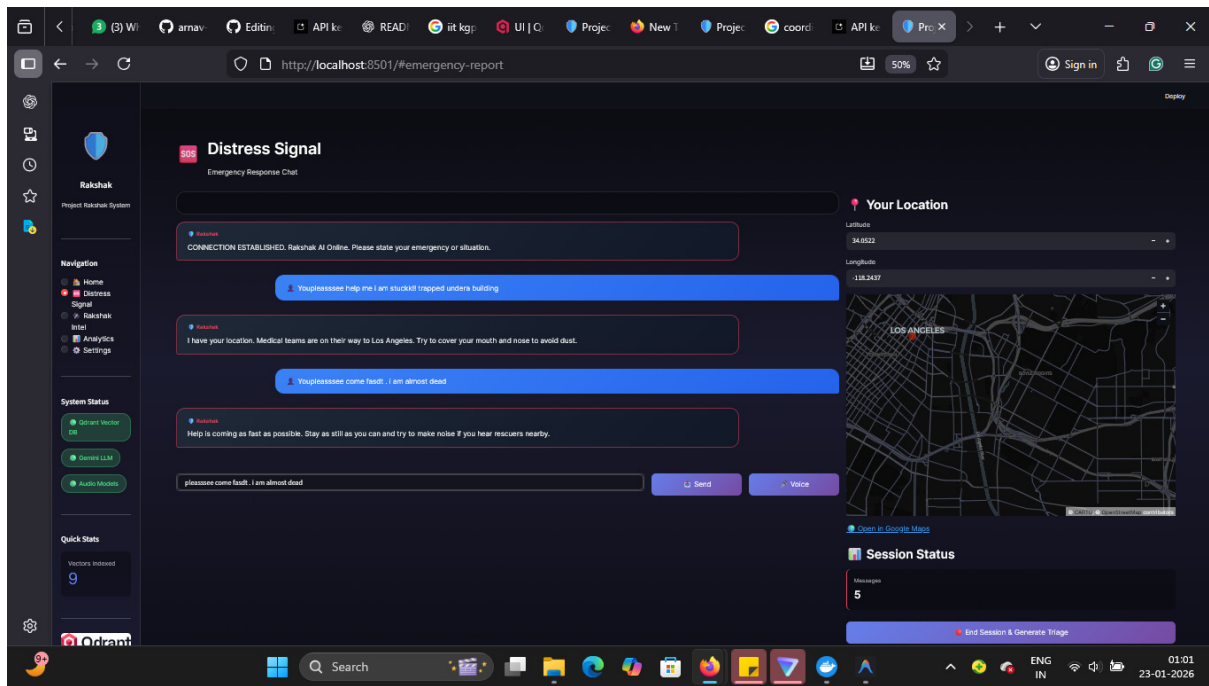


Figure 3: Distress Subsystem: Real-time victim interaction and panic analysis.

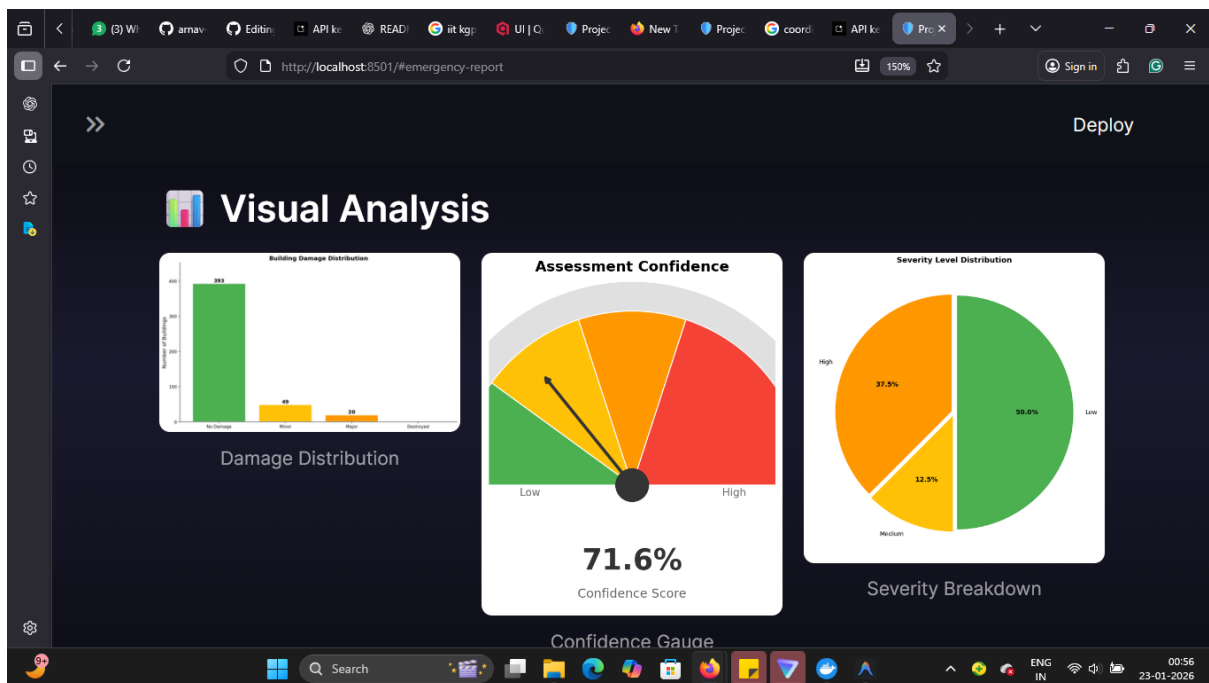


Figure 4: Analytics: System Performance Metrics.

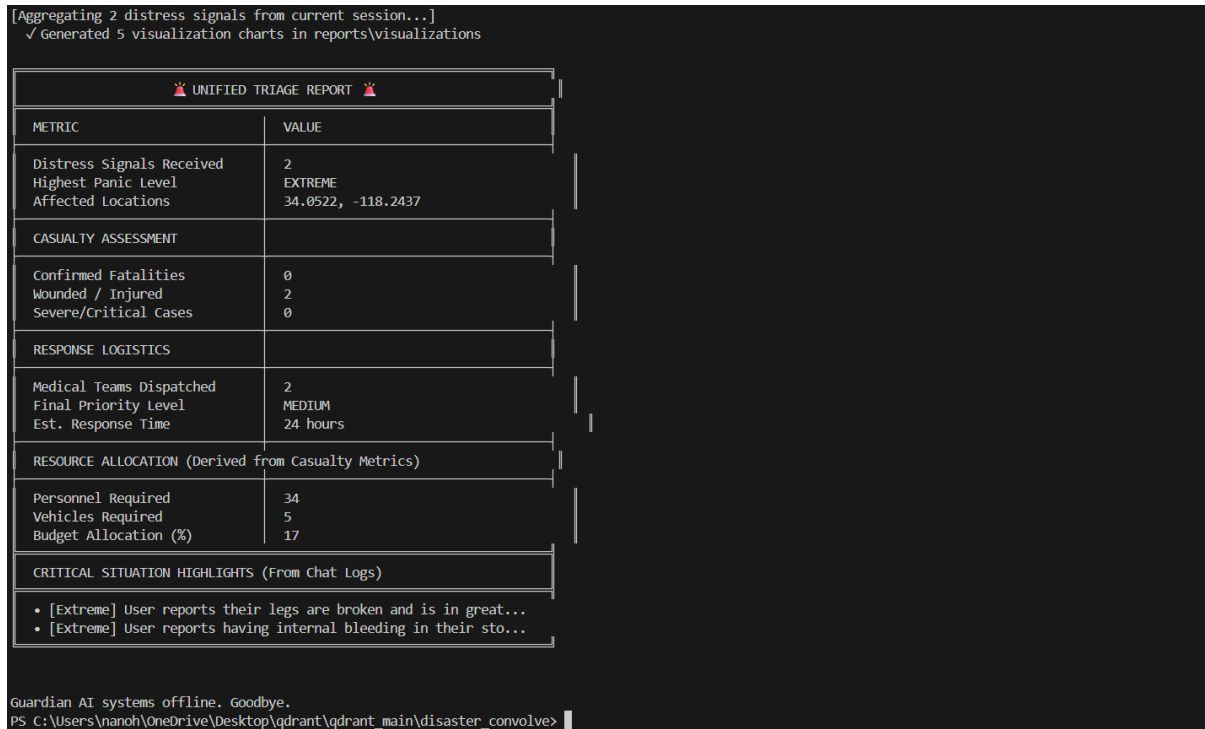


Figure 5: Final Unified Output: Fused command report.

Figures 2 through 5 display the system’s generated reports, from raw damage assessment to the final unified triage command.

7 Ethics, Safety and Novelty

Novelty and Comparison Unlike traditional disaster response systems that rely on manual GIS workflows or supervised CNNs (typically trained on a *single* event like a specific earthquake), Project Rakshak leverages **self-supervised DINOv2 embeddings** to achieve **Cross-Disaster Generalization**. It can transfer damage patterns learned from historical floods in Asia to assess new hurricanes in the Americas without retraining. While standard solutions are often unimodal (imagery only), our system introduces **multimodal fusion**, combining satellite data with acoustic distress signals (CLAP). Finally, we integrate a **dedicated Distress Agent** within an **active agentic architecture**, ensuring that satellite intelligence and victim-generated emergency reports are implemented together to trigger synchronized response team protocols and that too at a very **low latency** as compared to other systems..

Paradigm	Adaptability	Modality	Reasoning	Latency
Manual GIS Workflow	None (Manual)	Visual/Map	Human Expert	Hours
Supervised CNNs	Low (Retraining)	Visual Only	Fixed Class	Seconds
Project Rakshak (Ours)	High (Zero-Shot)	Visual+Audio+Text	Agentic+RAG	Seconds

Table 2: Comparison of traditional methods vs. Project Rakshak.

Ethical and Safety Considerations The system optimizes for **decision support**, not autonomous action. It inherits biases from the xBD dataset (e.g., building types). Strict PII

scrubbing and encryption are required for real-world deployment. The *Rakshak* persona is designed to be supportive but non-committal regarding physical rescue guarantees to avoid inducing false hope.

8 Conclusion

Project Rakshak demonstrates that replacing static training data with **Qdrant-backed episodic memory** enables AI systems to make grounded, explainable disaster assessments. By fusing satellite imagery with live human distress signals through a multi-agent architecture, the system offers a scalable blueprint for next-generation emergency response intelligence.

9 Future Scope

Given the efficiency gains from Binary Quantization, future iterations will focus on ****Edge Deployment on Drone Swarms****, allowing autonomous UAVs to perform this multimodal triage completely offline in severed-network zones.

Code Availability: The complete source code, including agent implementations and deployment instructions, is available on GitHub at: <https://github.com/arnav-chauhan-kgpian/project-rakshak>.

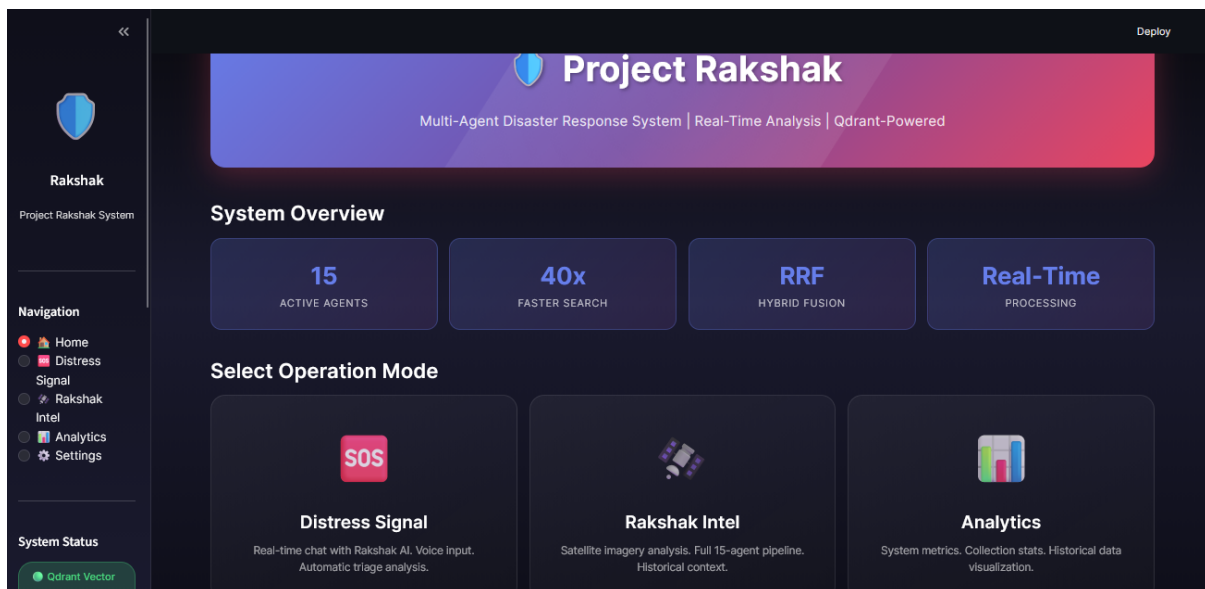


Figure 6: Final User Interface: As seen on Streamlit

A Agent Breakdown

The platform is architected as a set of 15 cooperating agents across three layers:

Layer 1 – Perception and Ingestion

- **SatelliteAgent:** Preprocesses raw imagery and synthesizes standardized Areas-of-Interest (AOI) for consistent analysis.
- **EmbeddingAgent:** Encodes visual features using a **DINOv2 ViT-B/14** foundation model to produce 768-dimensional dense vectors.
- **SparseEmbeddingAgent:** Tokenizes metadata into **BM25-weighted sparse vectors**, enabling precise keyword-based retrieval.
- **AudioIngestionAgent:** Transcribes emergency calls via **Whisper** and computes **CLAP embeddings** for audio panic analysis.
- **MetadataAgent:** Parses JSON labels to extract structured disaster metrics (damage counts, sensor consistency).
- **QdrantUpsertAgent:** Serializes multimodal data (Dense+Sparse+Payload) and commits it to the `disaster_memory` collection, ensuring robust data persistence via mmap/WAL to survive system restarts (Memory).

Layer 2 – Retrieval and Search

- **SearchProcessorAgent:** Constructs optimized hybrid queries and applies cross-disaster transfer penalties to filter irrelevant domains.
- **SearchExecutionAgent:** Executes parallel multimodal retrieval, fusing results via **Reciprocal Rank Fusion (RRF)** for stable ranking.

Layer 3 – Reasoning and Post-Processing

- **EvidenceSynthesisAgent:** Aggregates damage patterns and expands retrieval scope using Qdrant's **Recommendation API**.
- **GeoSimilarityAgent:** Performs fast **Geo-Radius** filtering to identify and retrieve spatially proximal historical precedents.
- **HistorySummarizerAgent:** Synthesizes retrieved incident reports into a coherent, evidence-backed historical narrative.
- **LLMReasoningAgent:** Orchestrates the **Gemini 2.5 Flash** model to generate the primary damage assessment and triage draft.
- **ReportAuditorAgent:** Self-critiques the draft report against ground-truth evidence to detect and correct hallucinations (Safety). This verification layer ensures that the system's output is factually accurate and safe for operational decision-making.
- **PostProcessorAgent:** Calibrates final confidence scores using ensemble consistency and generates severity visualizations.

Distress Subsystem

- **VictimChatAgent:** Manages real-time emergency communication, using **Whisper** for transcription and **CLAP** for panic level detection.

Coordinator

- **CentralCoordinator:** The DAG orchestrator that utilizes **multithreading** to manage data flow and parallel execution of the 15-agent pipeline.

B Implementation & Reproducibility

B.1 Methodology Details: Datasets

The system is evaluated using the **xBD (xView2) dataset** (19 major disasters).

- **Imagery:** Post-disaster satellite imagery (PNG) in `train/images/`.
- **Labels:** Geo-referenced JSON files in `train/labels/` containing building polygons and damage levels (*no-damage, minor, major, destroyed*).

Additional chat logs (`chat_logs/session_*.json`) and audio clips are used to validate the distress subsystem.

B.2 Technology Stack

The system is implemented in **Python 3.9+** with:

- **Qdrant Python Client:** For dense/sparse/geo retrieval.
- **DINOv2:** For visual feature extraction.
- **LLMs:** Gemini 2.5 Flash and OpenAI/Anthropic integrations.

B.3 Execution Instructions

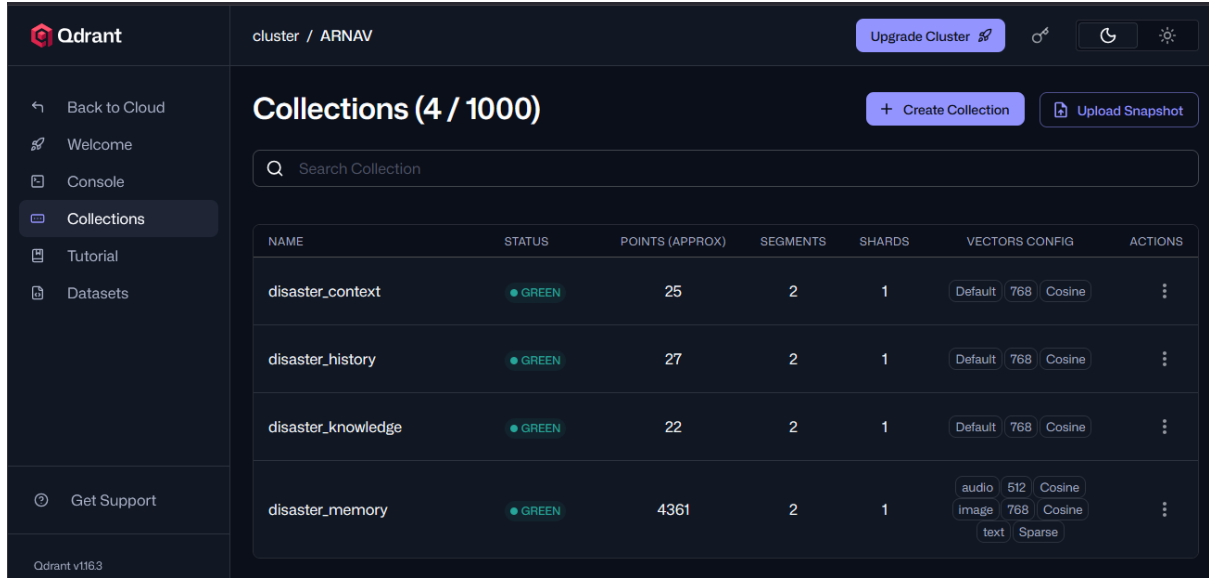
- `python main.py -reset:` Recreate Qdrant collections.
- `python main.py -batch-ingest:` Ingest xBD data.
- `python main.py:` Run single-incident Rakshak assessment.
- `python run_system.py:` Interactive Control Center for Distress Mode.

C Qdrant Collections Structure

This section visualizes the vector collections deployed on Qdrant Cloud. The system utilizes a sophisticated **multi-tier memory architecture** comprising 4 distinct collections:

1. **disaster_memory (Main Working Memory):** Stores the raw incident data (embeddings + metadata) for every processed disaster. This serves as the primary ingestion point.
2. **disaster_context (Short-Term / Session Memory):** Acts as the agent's RAM, storing recent interactions relative to a specific session. It has a **24-hour TTL** and decays rapidly to maintain relevance.

3. **disaster_history (Episodic Memory):** Maintains a log of system actions (reports generated, decisions made). It has a **30-day TTL**, allowing the system to recall past actions but forget them over time.
4. **disaster_knowledge (Semantic / Long-Term Memory):** Stores high-confidence patterns and learned facts (e.g., "Earthquakes in this region usually cause bridge failures"). This memory is **permanent** and constitutes the system's long-term learning.



NAME	STATUS	POINTS (APPROX)	SEGMENTS	SHARDS	VECTORS CONFIG	ACTIONS
disaster_context	GREEN	25	2	1	Default 768 Cosine	
disaster_history	GREEN	27	2	1	Default 768 Cosine	
disaster_knowledge	GREEN	22	2	1	Default 768 Cosine	
disaster_memory	GREEN	4361	2	1	audio 512 Cosine image 768 Cosine text Sparse	

Figure 7: Qdrant Cloud Dashboard: Active collections showing vector counts and optimization status.

D Research-Grade Technical Glossary

D.1 Embeddings & Representation Learning

DINOv2 (Dense Image Embeddings) We utilize **DINOv2 ViT-B/14**, a Vision Transformer (ViT) pre-trained via self-supervised learning (SSL). Unlike supervised models (e.g., ResNet trained on ImageNet), DINOv2 optimizes a combined objective of discriminatory self-supervision (DINO loss) and masked image modeling (iBOT loss).

- **Robustness:** It captures structural scene geometry and object permanence without class labels, making it superior for *out-of-distribution* (OOD) tasks like disaster damage assessment where labeled data is scarce.
- **Dimensionality:** It produces a $d = 768$ dense vector $\mathbf{v}_{img}$ which lies in a semantic manifold where Euclidean distance correlates with structural similarity.

CLAP (Contrastive Language-Audio Pretraining) For audio analysis, we forego simple decibel metering in favor of **CLAP**. This bi-encoder architecture maps audio waveforms and text captions into a shared latent space $\mathbb{R}^d$ ($d = 512$).

$$\text{sim}(A, T) = \frac{\mathbf{e}_A \cdot \mathbf{e}_T}{\|\mathbf{e}_A\| \|\mathbf{e}_T\|}$$

By computing the cosine similarity between the input audio embedding $\mathbf{e}_A$ and text anchor embeddings $\mathbf{e}_T$ (e.g., "screaming", "explosion", "calm voice"), we achieve robust, zero-shot panic detection.

BM25 (Sparse Embeddings) While dense vectors capture semantics, they often miss precise keyword matches (e.g., "7.2 magnitude"). We employ BM25 (Best Matching 25) to generate sparse vectors. The score for a query Q and document D is:

$$\text{BM25}(D, Q) = \sum_{q \in Q} \text{IDF}(q) \cdot \frac{f(q, D) \cdot (k_1 + 1)}{f(q, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

This allows the system to prioritize exact metadata matches alongside semantic visual matches.

D.2 Fusion Retrieval Algorithms

Reciprocal Rank Fusion (RRF) Combining dense (dot product) and sparse (BM25) scores is mathematically non-trivial due to disjoint score distributions. We strictly use RRF, a non-parametric rank aggregation method:

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + r(d)}$$

where $r(d)$ is the rank of document d in result set r , and k is a constant (typ. 60). This ensures a stable top-k retrieval without requiring heuristic weight tuning.

Binary Quantization (BQ) To achieve millisecond-latency search, we utilize Binary Quantization. We transform the floating-point vector $\mathbf{x} \in \mathbb{R}^d$ into a binary vector $\mathbf{b} \in \{0, 1\}^d$ via a sign function:

$$b_i = \begin{cases} 1 & \text{if } x_i > 0 \\ 0 & \text{otherwise} \end{cases}$$

Distance is then approximated using the **Hamming distance**, which can be computed via highly optimized XOR CPU instructions. This typically yields a $32\times$ memory reduction and $40\times$ speedup, with a negligible drop in recall (recovered via oversampling-and-rescoring).

D.3 HNSW (Hierarchical Navigable Small World)

The underlying Approximate Nearest Neighbor (ANN) index in Qdrant is HNSW. It constructs a multi-layer graph where lower layers capture fine-grained local connections and higher layers serve as an express highway for long-range navigation, allowing search complexity to scale logarithmically, $O(\log N)$.